

Database

Lecture 5: Advanced SQL

Yunyong Ko
School of Computer Science and Engineering
Chung-Ang University

Outline



- **Advanced SQL: Data Retrieval**
  - Nested Query
  - Joined Tables
  - Aggregation Functions and Grouping in SQL
- **Advanced SQL: More Functions**
  - Assertions and Triggers
  - Virtual Table: View

Nested Query

❑ What is a Nested Query?

- Complete SELECT-FROM-WHERE Blocks within a WHERE clause of another query
  - ❑ Useful to fetch existing values in a database and then use them for a comparison in a WHERE condition
- Returns a **(multi)set of tuples**

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

Outer query

```
SELECT    DISTINCT Pnumber
FROM      PROJECT
WHERE     Pnumber IN                                   Inner query
          ( SELECT    Pnumber
            FROM      PROJECT, DEPARTMENT, EMPLOYEE
            WHERE     Dnum = Dnumber AND
                      Mgr_ssn = Ssn AND Lname = 'Smith' )
          OR
          Pnumber IN                                   Inner query
          ( SELECT    Pno
            FROM      WORKS_ON, EMPLOYEE
            WHERE     Essn = Ssn AND Lname = 'Smith' );
```

Comparison Operator

❑ IN Operator

- Compares a value *v* with a set (or multiset) of values *V*
- Evaluates to **TRUE** if *v* is *one of the elements* in *V*

```
SELECT   DISTINCT Pnumber
FROM     PROJECT
WHERE    Pnumber IN
         ( SELECT   Pnumber
           FROM     PROJECT, DEPARTMENT, EMPLOYEE
           WHERE    Dnum = Dnumber AND
                    Mgr_ssn = Ssn AND Lname = 'Smith' )
         OR
         Pnumber IN
         ( SELECT   Pno
           FROM     WORKS_ON, EMPLOYEE
           WHERE    Essn = Ssn AND Lname = 'Smith' );
```

Is this *Pnumber* in V1?

Is this *Pno* in V2?

V1: A set of tuples

V2: A set of tuples

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

Comparison Operator (Cont.)

= ALL or ANY

- Evaluates to **TRUE** if value *v* is *equal to some (or all) value* in set *V*
- In addition to '=', other operators can be combined
  - >, >=, <, <=, <> (not equal)

Q. List the names of employees whose salary is greater than the salary of all the employees in department 5

```
SELECT   Lname, Fname
FROM     EMPLOYEE
WHERE    Salary > ALL
( SELECT   Salary
  FROM     EMPLOYEE
  WHERE    Dno = 5 );
```

The salaries of the employees in Department 5

Equivalent to IN operator

Salary **= ALL** **(SELECT …)**	Pnumber **= ANY** **(SELECT …)**
Salary **> ALL** **(SELECT …)**	Pnumber **> ANY** **(SELECT …)**
Salary **< ALL** **(SELECT …)**	Pnumber **< ANY** **(SELECT …)**
⋮	⋮

Ambiguity in Nested Queries

❑ Multiple Levels of Nested Queries

- Possible ambiguity among attribute names if same attribute names exist
  - ❑ One in FROM of the outer query, another in FROM of the nested query
- Specify an alias to avoid this ambiguity

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
Q16:  SELECT   E.Fname, E.Lname
      FROM     EMPLOYEE AS E
      WHERE    E.Ssn IN   ( SELECT   D.Essn
                            FROM     DEPENDENT AS D
                            WHERE    E.Fname = D.Dependent_name
                            AND      E.Sex = D.Sex );
```

It is generally advisable to create tuple variables (aliases) for *all the tables referenced in an SQL query* to avoid potential errors and ambiguities

Boolean Functions

❑ EXISTS and NOT EXISTS Function

- Check *whether the result of a nested query is empty or not*
  - ❑ Usually used in a **WHERE** clause condition
- Return **TRUE** if there are (no) tuples in the result of the nested query

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
SELECT   E.Fname, E.Lname
FROM     EMPLOYEE AS E
WHERE    EXISTS ( SELECT   *
                  FROM     DEPENDENT AS D
                  WHERE    E.Ssn = D.Essn AND E.Sex = D.Sex
                           AND E.Fname = D.Dependent_name);
```

Query 6. Retrieve the names of employees who have no dependents.

```
Q6:      SELECT   Fname, Lname
         FROM     EMPLOYEE
         WHERE    NOT EXISTS ( SELECT   *
                               FROM     DEPENDENT
                               WHERE    Ssn = Essn );
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Boolean Functions (Cont.)

When Boolean Functions Useful?

- Q. List the names of *managers who have at least one dependent*
  - **First nested query**: select all DEPENDENT tuples related to an EMPLOYEE
  - **Second nested query**: select all DEPARTMENT tuples managed by the EMPLOYEE

```
SELECT   Fname, Lname
FROM     EMPLOYEE
WHERE    EXISTS ( SELECT   *
                  FROM     DEPENDENT
                  WHERE    Ssn = Essn )
         AND
         EXISTS ( SELECT   *
                  FROM     DEPARTMENT
                  WHERE    Ssn = Mgr_ssn );
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

"Franklin, Wong" is retrieved

"Jennifer, Wallace" is retrieved

Exercises: Boolean Functions

❑ Exercise 1: Specify the Following Query

Q. Retrieve the name of each employee who works on all the projects controlled by department number 5

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

JOIN Tables in SQL

❑ What is a Joined Table?

- Table resulting from a join operation in the FROM clause
  - ❑ Mixing together all the selection and join conditions in WHERE clause

Q. Retrieve the name and address of all employees
who work for the 'Research' department

```
SELECT   Fname, Lname, Address
FROM     EMPLOYEE, DEPARTMENT
WHERE    Dname = 'Research' AND Dnumber = Dno;
```

=

```
SELECT   Fname, Lname, Address
FROM     (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE    Dname = 'Research';
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

+

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

First table | Second table

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno	Dname	Mgr_ssn	Mgr_start_date
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5	Research	333445555	1988-05-22
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5	Research	333445555	1988-05-22
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4	Administration	987654321	1995-01-01
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4	Administration	987654321	1995-01-01
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5	Research	333445555	1988-05-22
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5	Research	333445555	1988-05-22
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4	Administration	987654321	1995-01-01
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1	Headquarters	888665555	1981-06-19

The joined table have all the attributes of the first table (EMPLOYEE), followed by all the attributes of the second table (DEPARTMENT)

→ *The joined attributes are included only once in the resulting relation*

Natural Join

❑ What is a Natural Join?

- Creates an *implicit condition for each pair of attributes with the same name*
  - ❑ i.e., No join condition is specified

Q. Retrieve the name and address of all employees who work for the 'Research' department

```
SELECT   Fname, Lname, Address
FROM     (EMPLOYEE NATURAL JOIN
          (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE    Dname = 'Research';
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

+

DEPARTMENT

Dname	Dno	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

First table | Second table

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno	Dname	[redacted]	Mgr_ssn	Mgr_start_date
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5	Research		333445555	1988-05-22
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5	Research		333445555	1988-05-22
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4	Administration		987654321	1995-01-01
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4	Administration		987654321	1995-01-01
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5	Research		333445555	1988-05-22
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5	Research		333445555	1988-05-22
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4	Administration		987654321	1995-01-01
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1	Headquarters		888665555	1981-06-19

The joined table have all the attributes of the first table (EMPLOYEE), followed by all the attributes of the second table (DEPARTMENT)

→ *The joined attributes are included only once in the resulting relation*

Inner Join

❑ What is Inner Join?

- Default type of join is called an **inner join**,
  - ❑ A tuple is included in the result only if a matching tuple exists in the other relation

```
SELECT   E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM     EMPLOYEE AS E, EMPLOYEE AS S
WHERE    E.Super_ssn = S.Ssn;
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

In the inner join,
only pairs of tuples that match the join condition are retrieved

Ssn	Super_ssn
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321

Only employees who have a supervisor are retrieved.
A tuple whose value for Super_ssn is NULL is excluded.

Q. What if we want that all employees are retrieved?

→ OUTER JOIN!

Outer Join

What is Outer Join?

- To retrieve tuples *if it does not have a matching tuple*
  - It is padded with NULL values for the attributes

```
SELECT   E.Lname AS Employee_name,
         S.Lname AS Supervisor_name
FROM     (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
         ON E.Super_ssn = S.Ssn);
```

EMPLOYEE

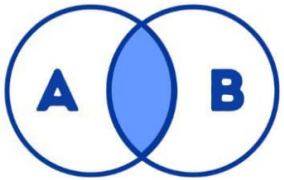
Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Ssn	Super_ssn
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321
888665555	NULL

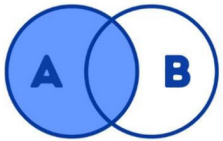
In LEFT OUTER JOIN,
every tuple in the left table must appear in the result.

Visualization of Inner and Outer Joins

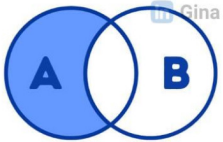
❑ Visualized Different Types of Joins



Inner join

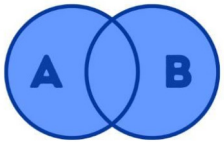


SELECT * FROM
A LEFT JOIN B
ON A.KEY = B.KEY

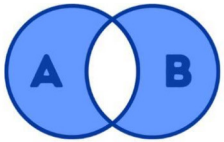


SELECT * FROM
A LEFT JOIN B
ON A.KEY = B.KEY
WHERE B.KEY IS NULL

LEFT OUTER JOIN

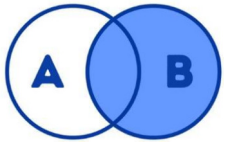


SELECT * FROM
A FULL OUTER JOIN B
ON A.KEY = B.KEY

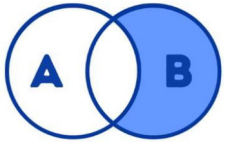


SELECT * FROM
A FULL OUTER JOIN B
ON A.KEY = B.KEY WHERE A.KEY
IS NULL OR B.KEY IS NULL

FULL OUTER JOIN



SELECT * FROM
A RIGHT JOIN B
ON A.KEY = B.KEY



SELECT * FROM
A RIGHT JOIN B
ON A.KEY = B.KEY
WHERE A.KEY IS NULL

RIGHT OUTER JOIN

Aggregate Functions in SQL

What is an Aggregate function?

- Summarize information from multiple tuples into a ***single-tuple summary***
  - Usually used in the **SELECT** (or HAVING) clause

Built-In Aggregate Functions

- **COUNT**
  - Returns *the number of tuples or values* as specified in a query
- **SUM**, **MAX**, **MIN**, and **AVG**
  - Can be applied to a set or multiset of numeric values
  - Return *the sum, maximum value, minimum value, and average* of those values

Aggregate Functions in SQL (Cont.)

❑ SUM, MAX, MIN, and AVG Functions

- Can be applied to a set or multiset of numeric values
- Return *the sum, maximum value, minimum value, and average* of those values

Query 19. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
Q19:    SELECT    SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
        FROM      EMPLOYEE;
```

Query 20. Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20:    SELECT    SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
        FROM      (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
        WHERE     Dname = 'Research';
```

It returns a single-row summary of all the rows in the EMPLOYEE table

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Aggregate Functions in SQL (Cont.)

❑ COUNT Function

- Counting *the number of tuples* as specified in a query

Queries 21 and 22. Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
Q21:  SELECT  COUNT (*)
      FROM    EMPLOYEE;

Q22:  SELECT  COUNT (*)
      FROM    EMPLOYEE, DEPARTMENT
      WHERE   DNO = DNUMBER AND DNAME = 'Research';
```

The asterisk (*) refers to the rows (tuples),
COUNT (*) returns the number of rows in the result of the query.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

- Counting *the number of values* as specified in a query
  - ❑ NULL values are not counted

Query 23. Count the number of distinct salary values in the database.

```
Q23:  SELECT  COUNT (DISTINCT Salary)
      FROM    EMPLOYEE;
```

Aggregate Functions in SQL (Cont.)

❑ Other Usage of Aggregate Functions

- Can be used in selection conditions involving nested queries as well
  - ❑ (1) Specify a nested query with an aggregate function
  - ❑ (2) Use the nested query result in the WHERE clause of an outer query

Q. Retrieve the names of all employees who have two or more dependents

```
SELECT   Lname, Fname
FROM     EMPLOYEE
WHERE    ( SELECT   COUNT (*)
           FROM     DEPENDENT
           WHERE    Ssn = Essn ) >= 2;
```

→ Counting the number of dependents

Grouping in SQL

What is Grouping?

- Partition non-overlapping subsets (**groups**) of tuples in a relation
  - We may want to apply the aggregate functions to *groups of tuples in a relation*
    - e.g., Find the average salary of employees *in each department* or the number of employees who work *on each project*
- Useful when we want to summarize those groups independently

Grouping Attributes

- Attributes used as the criteria for grouping
  - Each group consists of the tuples that have *the same value for grouping attribute(s)*
- Specified by **GROUP BY** clause

Query 24. For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
Q24:    SELECT     Dno, COUNT (*), AVG (Salary)
        FROM       EMPLOYEE
        GROUP BY   Dno;
```

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

The EMPLOYEE tuples are partitioned into groups,
Each group has the same group attribute value (Dno)

Grouping in SQL (Cont.)

❑ Example of Grouping in the COMPANY Database

- Q. For each project, retrieve the project number, the project name, and the number of employees who work on that project

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

```
SELECT    Pnumber, Pname, COUNT (*)
FROM      PROJECT, WORKS_ON
WHERE     Pnumber = Pno
GROUP BY  Pnumber, Pname;
```

Pname	Pnumber	···	Essn	Pno	Hours	
ProductX	1		123456789	1	32.5	2
ProductX	1		453453453	1	20.0	
ProductY	2		123456789	2	7.5	3
ProductY	2		453453453	2	20.0	
ProductY	2		333445555	2	10.0	
ProductZ	3		666884444	3	40.0	2
ProductZ	3		333445555	3	10.0	
Computerization	10	···	333445555	10	10.0	3
Computerization	10		999887777	10	10.0	
Computerization	10		987987987	10	35.0	
Reorganization	20		333445555	20	10.0	3
Reorganization	20		987654321	20	15.0	
Reorganization	20		888665555	20	NULL	
Newbenefits	30		987987987	30	5.0	3
Newbenefits	30		987654321	30	20.0	
Newbenefits	30		999887777	30	30.0	

1, ProductX, 2

2, ProductY, 3

3, ProductZ, 2

⋮

Grouping in SQL (Cont.)

❑ HAVING Clause

- Specify a "***condition***" on *the summary information regarding the group of tuples*
- Can appear in conjunction with a GROUP BY clause

Query 26. For each project *on which more than two employees work,* retrieve the project number, the project name, and the number of employees who work on the project.

```
Q26:   SELECT     Pnumber, Pname, COUNT (*)
       FROM       PROJECT, WORKS_ON
       WHERE      Pnumber = Pno
       GROUP BY   Pnumber, Pname
       HAVING     COUNT (*) > 2;
```

Only groups satisfying this condition are retrieved

Pname	Pnumber	...	Essn	Pno	Hours
ProductX	1		123456789	1	32.5
ProductX	1		453453453	1	20.0
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
ProductZ	3		666884444	3	40.0
ProductZ	3		333445555	3	10.0
Computerization	10	...	333445555	10	10.0
Computerization	10		999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

Pname	Pnumber	...	Essn	Pno	Hours
ProductY	2		123456789	2	7.5
ProductY	2		453453453	2	20.0
ProductY	2		333445555	2	10.0
Computerization	10		333445555	10	10.0
Computerization	10	...	999887777	10	10.0
Computerization	10		987987987	10	35.0
Reorganization	20		333445555	20	10.0
Reorganization	20		987654321	20	15.0
Reorganization	20		888665555	20	NULL
Newbenefits	30		987987987	30	5.0
Newbenefits	30		987654321	30	20.0
Newbenefits	30		999887777	30	30.0

After applying the HAVING clause condition

Pname	Count (*)
ProductY	3
Computerization	3
Reorganization	3
Newbenefits	3

Result of Q26 (Pnumber not shown)

Exercises: Aggregate Functions

❑ Exercise 2: Specify the following query in SQL

Q. For each department whose average employee salary is more than $30,000, retrieve the department name and the number of employees working for that department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Outline

- Advanced SQL: Data Retrieval
  - Nested Query
  - Joined Tables
  - Aggregation Functions and Grouping in SQL

- **Advanced SQL: More Functions**
  - Assertions and Triggers
  - Virtual Table: View



Specifying General Constraints & Actions

❑ Assertions

- Specifying general constraints that do not fall into any of the built-in constraints
  - ❑ i.e., not covered by the key constraint, entity and referential integrity constraints
  - ❑ e.g., *the salary of an employee must not be greater than the salary of the manager of the department*

❑ Triggers

- Specifying automatic actions to be taken when certain events and conditions occur
  - ❑ e.g., a manager may want to be informed *if an employee's travel expenses exceed a certain limit*



EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Assertions



Triggers

Specifying Constraints as Assertions

❑ The CREATE ASSERTION Command

- Specifying a general constraint that users want to check
- Example: SALARY_CONSTRAINT
  - **CHECK** is followed by a ***condition*** in parentheses() that *must hold true on every database state*
    - Constraints are usually specified using the **EXISTS** and **NOT EXISTS** style of SQL conditions

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT     *
                     FROM       EMPLOYEE E, EMPLOYEE M,
                                DEPARTMENT D
                     WHERE      E.Salary>M.Salary
                     AND        E.Dno = D.Dnumber
                     AND        D.Mgr_ssn = M.Ssn ) );
```

→ **True**: the result is empty
→ **False**: the result is not empty

Specifying *a query that selects any tuples that violate the desired condition* with a **NOT EXISTS** clause

Specifying Actions as Triggers

❑ The CREATE TRIGGER Command

- Specifying the type of action to be taken when certain conditions are satisfied
- Example: SALARY_VIOLATION
  - ❑ We want to check whenever an employee's salary is greater than the salary of his/her direct supervisor
  - ❑ Several events can trigger this rule
    - Inserting a new employee tuple, changing an employee's salary, or changing an employee's supervisor
  - ❑ The action to take is to call an external stored procedure SALARY_VIOLATION
- A trigger is regarded as an **ECA (Event, Condition, Action) rule**

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

Triggers can be used in various applications,
such as (1) maintaining database consistency, (2) monitoring database updates, and (3) updating derived data automatically

Specifying Actions as Triggers: Event

❑ Three Components of Triggers: Event

- Update operations explicitly applied to the database
  - ❑ E.g, Inserting a new employee tuple, changing an employee's salary, or changing an employee's supervisor
- The events are specified with the keyword **BEFORE** or **AFTER**
  - ❑ The trigger should be executed **before/after** the triggering operation is executed
  - ❑ All possible events should be considered

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

Specifying Actions as Triggers: Condition

❑ Three Components of Triggers: Condition

- Determines whether the action should be executed when a triggering event occurs
  - ❑ Once the triggering event has occurred, an *optional condition* may be evaluated
- The condition is specified in **WHEN clause** of the trigger
  - ❑ Only if it evaluates to true, the action will be executed

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

Specifying Actions as Triggers: Action

❑ Three Components of Triggers: Action

- The rule action to be executed automatically
- The action is a sequence of SQL statements, or a database transaction, or an external program

```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                         WHERE SSN = NEW.SUPERVISOR_SSN ) )
                         INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                         NEW.Ssn );
```

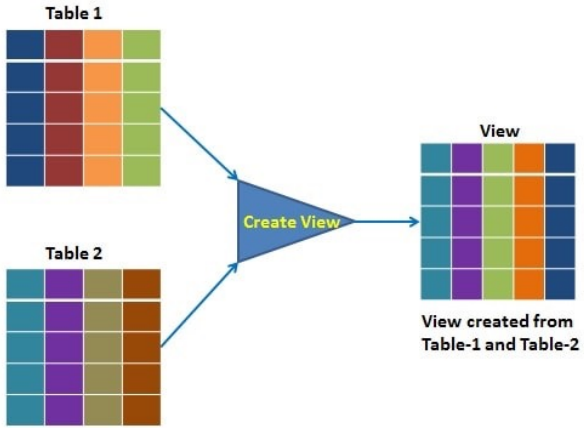
Virtual Table: View

What is a View in SQL?

- A ***virtual*** table ***derived*** from base tables
  - Not necessarily exist in a physical form

Operations on a View

- Retrieval operations on a view does not have any limitations
- Update operations on a view are usually limited (i.e., a view is '***read-only***')



Motivation of Views

❑ When to Use Views? → *To Simplify Complex Queries!*

- When frequently used queries reference multiple tables with complex conditions
  - ❑ e.g., join operations, aggregate functions, …

```
SELECT   Fname, Lname, Pname, Hours
FROM     EMPLOYEE, PROJECT, WORKS_ON
WHERE    Ssn = Essn AND Pno = Pnumber;
```

→ Whenever we execute this query, three tables are joined

```
SELECT   Dname, COUNT (*), SUM (Salary)
FROM     DEPARTMENT, EMPLOYEE
WHERE    Dnumber = Dno
GROUP BY Dname;
```

→ Whenever we execute this query, two tables are joined and aggregate functions are applied

What if we create a view?

WORKS_ON1

Fname	Lname	Pname	Hours

DEPT_INFO

Dept_name	No_of_emps	Total_sal

A View can be specified as the result of the joins

→ We can retrieve the same results by a single-table query

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

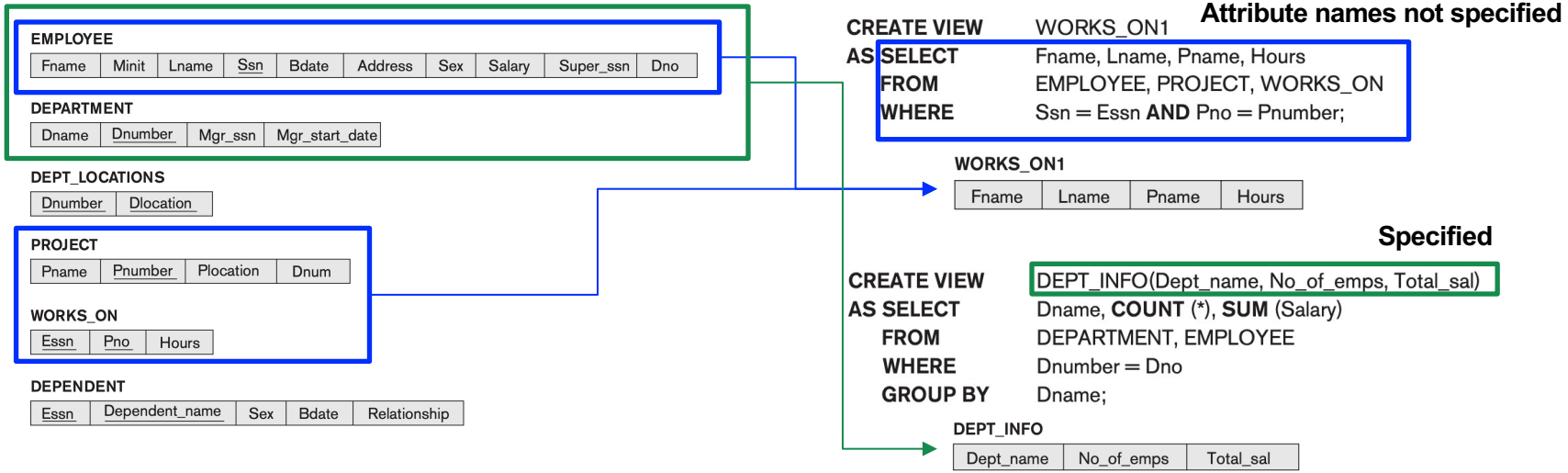
DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Specification of Views in SQL

❑ The CREATE VIEW Command

- **CREATE VIEW** <View Name> **AS** <Select Query>
  - ❑ If attribute names for the view are not specified, they would be the same as those of the defining tables
  - ❑ Attribute names can be specified explicitly



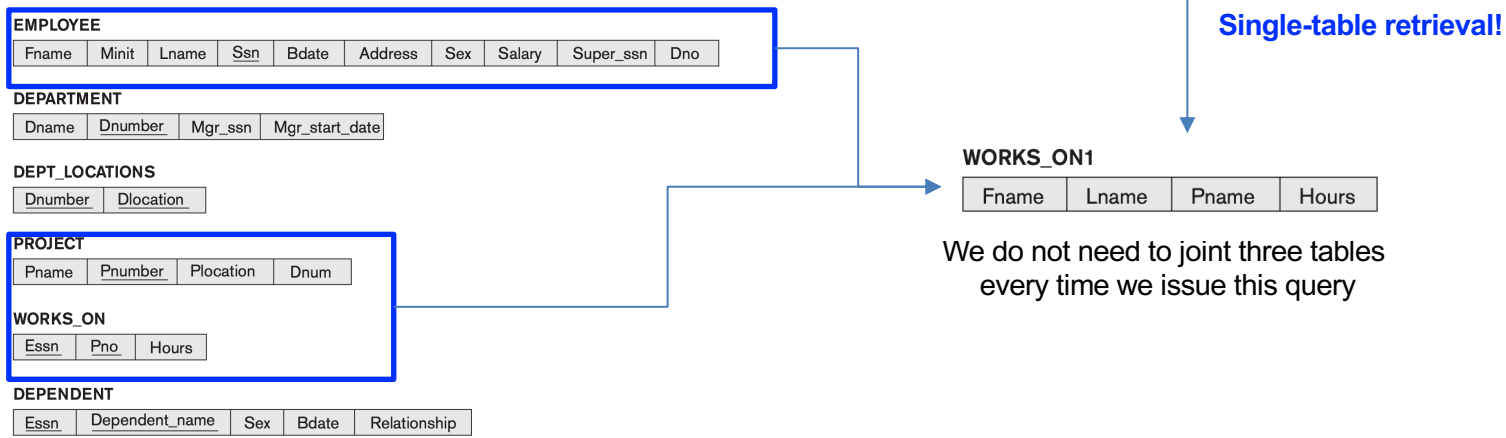
Specifying Queries on a View

❑ Data Retrieval from Views

- Specifying queries on a view in the same way as we do on a base table

Q. Retrieve the last name and first name of all employees who work on the 'ProductX' project

```
SELECT   Fname, Lname
FROM     WORKS_ON1
WHERE    Pname = 'ProductX';
```

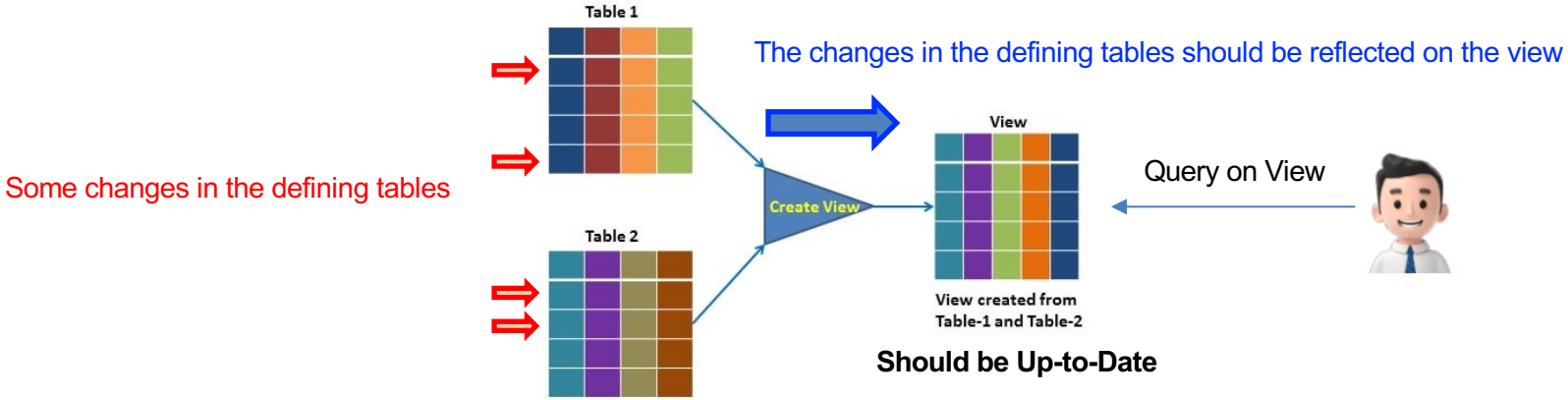


Characteristics of Views

❑ A View Should Be Up-to-Date

- What if the tuples in the defining tables are updated?
- DBMS is responsible for reflecting the changes to the view

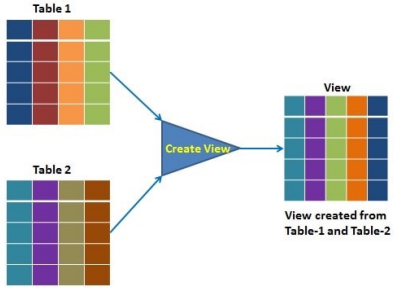
Q. How can the DBMS keep a view up-to-date?



View Implementation in the DBMS

Two Approaches

- Query modification
- View materialization



Query Modification (Approach 1)

- Transforming the view query into a query on the underlying base tables
  - Not maintaining views physically

```
CREATE VIEW    WORKS_ON1
AS SELECT      Fname, Lname, Pname, Hours
   FROM        EMPLOYEE, PROJECT, WORKS_ON
   WHERE       Ssn = Essn AND Pno = Pnumber;

   SELECT      Fname, Lname
   FROM        WORKS_ON1
   WHERE       Pname = 'ProductX';
```

Transforming the view query

```
SELECT    Fname, Lname
FROM      EMPLOYEE, PROJECT, WORKS_ON
WHERE     Ssn = Essn AND Pno = Pnumber
          AND Pname = 'ProductX';
```

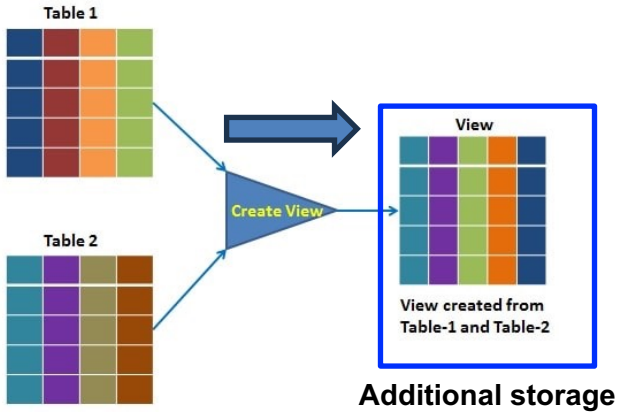
→ Not aligned with the philosophy of Views in SQL

Can be convenient to users,
But not to the DB system

View Implementation in the DBMS (Cont.)

❑ View Materialization (Approach 2)

- Physically create a view table when the view is first queried
- Keep that a virtual table "***for a while***" (i.e., *how long?*)
  - **Assumption**: *other retrieval queries on the view will follow frequently (hopefully)*
- **Incremental updates**
  - When an update is applied to ***one of the defining tables,*** it should be reflected to the view *incrementally*

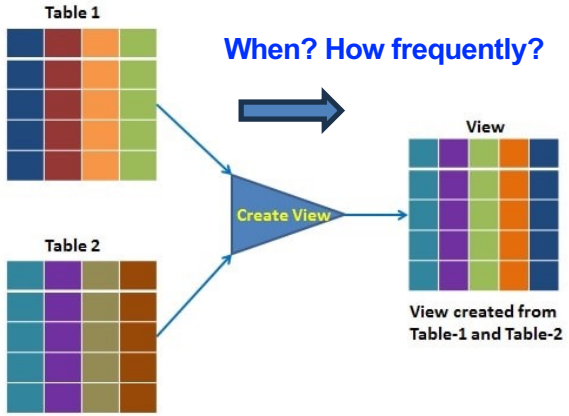


Q. When the defining tables are updated,
How/when to update the view?

View Implementation in the DBMS (Cont.)

When/How to Update the View?

- **Immediate update**
  - Update a view *as soon as the defining base tables are updated*
  - Always up-to-date, but sometime unnecessary and inefficient
- **Periodic update**
  - Updates a view *periodically*
  - A view query may get a result that is not up-to-date
- **Lazy update**
  - Update a view when it is needed by a view query
  - Not up-to-date until a query is specified on the view

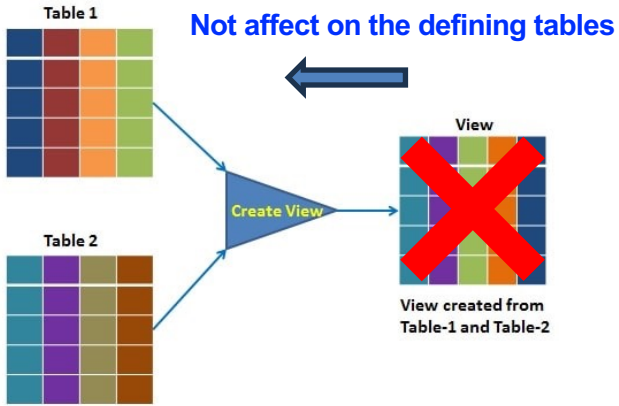


Dropping a View in SQL

❑ The DROP VIEW Command

- We can use the **DROP VIEW** command if we do not need a view anymore
  - The result of this command does not affect on its defining tables

```
DROP VIEW      WORKS_ON1;
```



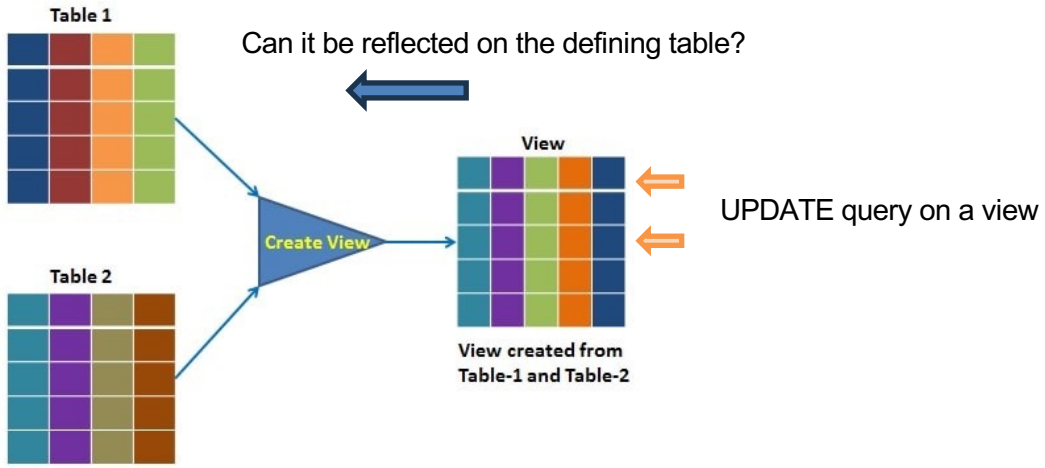
Q. What if the view is not queried for a certain period of time?
→ Remove the view

Q. What if the view is needed in the future?
→ Recompute it from scratch

Update Operations on Views

❑ Can a View be Updated?

- We can always issue any ***retrieval queries*** against any views
- INSERT, DELETE, or UPDATE command on a view is not possible in many cases



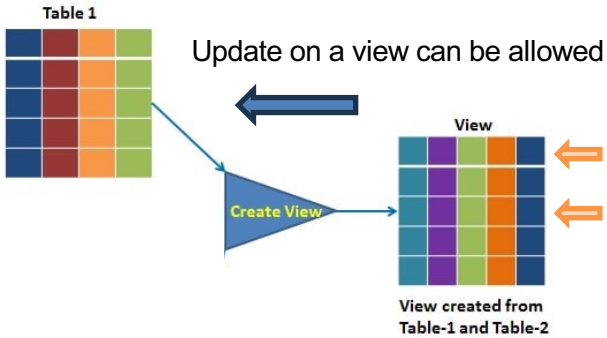
Two Cases of View Updates

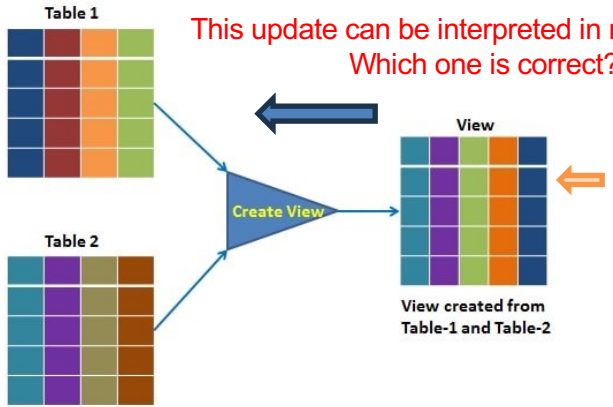
❑ Case 1 (Allowed)

- Update on a view defined on a single table without any aggregate functions
  - ❑ It can be mapped to an update on the underlying base table

❑ Case 2 (Not Allowed)

- Update on a view involving joins (i.e., multiple tables) or aggregation functions
  - ❑ It may be mapped to update operations on the underlying base relations in ***multiple ways***



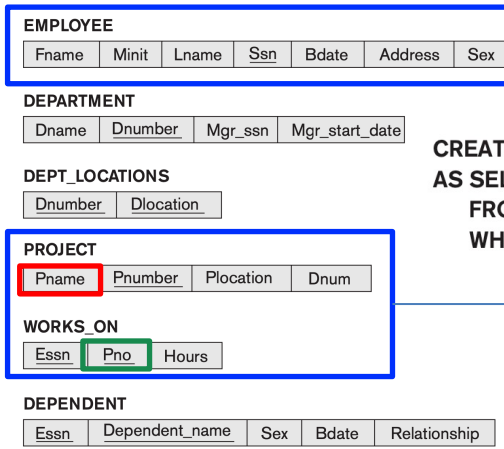


Case 2: Potential Problem of View Update

❑ Example: Update on WORKS_ON1 View

- Consider the WORKS_ON1 view, suppose that we issue the command:

```
UPDATE WORKS_ON1
SET    Pname = 'ProductY'
WHERE  Lname = 'Smith' AND Fname = 'John'
       AND Pname = 'ProductX';
```




```
CREATE VIEW   WORKS_ON1
AS SELECT     Fname, Lname, Pname, Hours
   FROM       EMPLOYEE, PROJECT, WORKS_ON
   WHERE      Ssn = Essn AND Pno = Pnumber;
```

WORKS_ON1

Fname	Lname	Pname	Hours
John	Smith	**ProductX**	17
John	Smith	**ProductY**	17

Q. What is the purpose of this update?

Update the name of the project that "John Smith" is working on

VS

Update the project name "ProductX" to "ProductY"

Case 2: Potential Problem of View Update

❑ Multiple Interpretations

- It can be mapped into several updates to give ***the desired update effect on the view***

```
UPDATE WORKS_ON1
SET    Pname = 'ProductY'
WHERE  Lname = 'Smith' AND Fname = 'John'
       AND Pname = 'ProductX';
```

Update the name of the project that "John Smith" is working on

WORKS_ON

Essn	Pno	Hours

```
UPDATE WORKS_ON
SET    Pno =     ( SELECT Pnumber
                   FROM   PROJECT
                   WHERE  Pname = 'ProductY' )
WHERE  Essn IN   ( SELECT Ssn
                   FROM   EMPLOYEE
                   WHERE  Lname = 'Smith' AND Fname = 'John' )
       AND
       Pno =     ( SELECT Pnumber
                   FROM   PROJECT
                   WHERE  Pname = 'ProductX' );
```

This relates 'John Smith' to the 'ProductY' PROJECT tuple, instead of the 'ProductX' PROJECT tuple

Update the name of the 'ProductX' tuple in the PROJECT relation to 'ProductY'

PROJECT

Pname	Pnumber	Plocation	Dnum

```
UPDATE PROJECT   SET    Pname = 'ProductY'
WHERE  Pname = 'ProductX';
```

This also give the desired update effect on the view, but additional side effects affecting the result of other queries

→ Changing all the view tuples with Pname='ProductX'

Case 2: Potential Problem of View Update

❑ Impossible Update

- Some view updates may not make much sense

```
UPDATE    DEPT_INFO
SET       Total_sal = 100000
WHERE     Dname = 'Research';
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

```
CREATE VIEW    DEPT_INFO(Dept_name, No_of_emps, Total_sal)
AS SELECT      Dname, COUNT (*), SUM (Salary)
    FROM       DEPARTMENT, EMPLOYEE
    WHERE      Dnumber = Dno
    GROUP BY   Dname;
```

The value computed by an aggregate function

DEPT_INFO

Dept_name	No_of_emps	Total_sal
Resaerch	10	**95,000**

Not stored physically

Resaerch	10	**100,000**

Q. What does this update mean?

View Update: Observations

❑ View Update

- View update is feasible when only ***one possible update*** on the base relations can accomplish the desired update operation on the view

❑ Observations

- **Case 1**: Multiple Tables with Joins
  - Views defined on multiple tables using joins are generally *not updatable*
    - Since a query specified on this view can be mapped to more than one update
- **Case 2**: Grouping and Aggregate Functions
  - Views defined using grouping and aggregate functions are *not updatable*
    - Since a query specified on this view does not make sense
- **Case 3**: Single Table
  - A view with a single defining table is *updatable* if the view attributes contain:
    - 1) The ***primary key*** of the base relation;
    - 2) All attributes with the ***NOT NULL*** constraint that ***do not have default values*** (→ why?)

Views as Authorization

❑ Authorization Mechanisms using Views

- Views can be used to hide **certain attributes** or **tuples** from unauthorized users.
- Example 1: DEPT5EMP
  - ❑ The manager of department 5 is only allowed to see information for employees working for department 5
    - Create the view 'DEPT5EMP' and grant the user the privilege to query the view but not EMPLOYEE
- Example 2: BASIC_EMP_DATA
  - ❑ Similarly, a view can restrict a user to only see certain columns
  - ❑ Normal employees can see only the basic data: first name, last name, and address of an employee

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE VIEW   DEPT5EMP   AS
SELECT        *
FROM          EMPLOYEE
WHERE         Dno = 5;

CREATE VIEW   BASIC_EMP_DATA   AS
SELECT        Fname, Lname, Address
FROM          EMPLOYEE;
```

Exercise: Views in SQL

❑ Exercise 3

- Specify the following view in SQL on the COMPANY database schema
- Explain when the view can be useful (i.e., guess the purpose)

A view that has the department name, manager name, and manager salary for every department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

Exercise: Views in SQL

❑ Exercise 4

- Specify the following view in SQL on the COMPANY database schema
- Explain when the view can be useful (i.e., guess the purpose)

A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

Takeaways: Advanced DML

- **Nested Query**
  - Complete SELECT-FROM-WHERE Blocks within a WHERE clause of another query
- **Joined Table**
  - Inner and outer joins
- **Aggregate Functions**
  - Summarize information from multiple tuples into a single-tuple summary
- **Grouping**
  - Partition non-overlapping subsets (**groups**) of tuples in a relation

SELECT <attribute and function list>
FROM <table list>
[**WHERE** <condition>]
[**GROUP BY** <grouping attribute(s)>]
[**HAVING** <group condition>]
[**ORDER BY** <attribute list>];

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Execution order

Takeaways: Advanced Functions

❑ Specifying General Constraints as Assertions

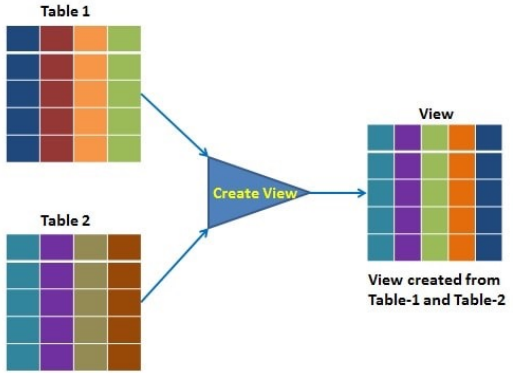
- User-defined constraints that outside the scope of the built-in constraints

❑ Specifying General Actions as Triggers

- User-defined type of action to be taken when certain events occur

❑ Virtual Table: View

- A virtual table derived from base tables
- A view should be up-to-date
  - Immediate update
  - Periodic update
  - Lazy update
- View update is feasible only when
  - One update is the only desired update



Q&A:
Advanced SQL